An Industry Oriented Mini Project Report

On

Advertisement Click Prediction

Submitted in Partial Fulfillment of the Academic Requirement for the Award of Degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

Submitted By:

CH. VYSHNAVI	**(23R01A05T7)**
K. PRANITHA	**(23R01A05V6)**
K. VASAVI	**(23R01A05V7)**

Under the esteemed guidance of

Mr. P. Krishna Reddy

Assistant Professor

Dept. of Computer Science and Engineering



CMR INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

Approved by AICTE, Permanent Affiliation to JNTUH, Accredited by NBA and NAAC, Kandlakoya(V), Medchal Dist - 501 401

www.cmrithyderabad.edu.in

2025-26

CMR INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

Approved by AICTE, Permanent Affiliation to JNTUH, Accredited by NBA and NAAC,
Kandlakoya(V), Medchal Dist - 501 401
www.cmrithyderabad.edu.in

CERTIFICATE

This is to certify that Industry Oriented Mini Project Report entitled with "Advertisement Click Prediction " is being submitted by:

CH. VYSHNAVI	**(23R01A05T7)**
K. PRANITHA	**(23R01A05V6)**
K. VASAVI	**(23R01A05V7)**

To JNTUH, Hyderabad, in partial fulfillment of the requirement for award of the degree of B.Tech in CSE (Data Science) and is a record of a bonafide work carried out under our guidance and supervision. The results in this project have been verified and are found to be satisfactory. The results embodied in this work have not been submitted to have any other University for award of any other degree or diploma.

GUIDE

Mr P. Krishna Reddy
Assistant Professor
Dept. of Computer Science and Engineering

HEAD OF THE DEPARTMENT

Dr K Pradeep Reddy
Professor & HOD
Dept. of Computer Science and Engineering

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We are extremely grateful to **Dr M Janga Reddy**, Director, **Dr B Satyanarayana**, Principal and **Dr K Pradeep Reddy**, Head of Department, Dept of Computer Science and Engineering, CMR Institute of Technology for their inspiration and valuable guidance during entire duration.

We are extremely thankful to **Mrs G. Lavanya,** Industry Oriented Mini Project Coordinator and internal guide **Mr P. Krishna Reddy**, Dept of Computer Science and Engineering, CMR Institute of Technology for their constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if we do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and coordination extended in bringing out this Project successfully in time.

Finally, we are very much thankful to our parents and relatives who guided directly or indirectly for every step towards success.

CH. VYSHNAVI (23R01A05T7)
K. PRANITHA (23R01A05V6)
K. VASAVI (23R01A05V7)

ABSTRACT

This project presents an Advertisement Click-Through Rate Prediction and Recommendation System that helps advertisers identify the probability of users clicking on a particular advertisement. The system mainly focuses on improving advertisement efficiency by analyzing user behavior data, browsing history, and advertisement details. After preprocessing the data and extracting useful features, the cleaned data is used to train a machine learning model for CTR prediction. To achieve better prediction accuracy, the DeepFM algorithm is used, as it effectively combines factorization machines with deep learning techniques. Based on the predicted click-through rates, the system also recommends target users whose CTR values are above a certain threshold, helping advertisers reach the right audience more efficiently.The proposed system is highly useful for online advertising platforms, e-commerce websites, and search engines where user interests play a major role in ad placement. By predicting the click probability in advance, advertisers can make better business decisions, improve conversion rates, and maximize revenue. At the same time, users are shown more relevant advertisements, which improves their browsing experience and reduces unnecessary ad exposure. Overall, the system provides an efficient, scalable, and intelligent solution for advertisement prediction and recommendation.

TABLE OF CONTENTS

1. Introduction	1
2. Literature Survey	2
3. System Analysis	7
3.1 Existing System	7
3.2 Disadvantages of Existing System	7
3.3 Proposed System	8
3.4 Advantages of Proposed System	9
3.5 Limitations of Advanced System	10
4. System Study	10
4.1 Feasibility Study	10
4.1.1 Economical Feasibility	10
4.1.2 Operational Feasibility	11
4.1.3 Technical Feasibility	12
5. Hardware and Software Requirements	13
5.1 Hardware Requirements	13
5.2 Software Requirements	14
6. Software Requirement Specification	15
7. Architecture	18
8. Modules	24
9. Diagrams	26
9.1 Class Diagrams	26
9.2 Use Case Diagrams	26
9.3 Sequence Diagrams	27
9.4 Collaboration Diagrams	28
9.5 Component Diagrams	29
9.6 Deployment Diagrams	20
9.7 Activity Diagrams	21
9.8 Data Flow Diagram	32
10. Implementation	33
11. Screenshots	38
12. Testing	41
13. Conclusion	46
14. References	47

LIST OF FIGURES

Figure No.	Particulars	Page No.
1	SDLC Umbrella Model Diagram	18
2	Requirements Gathering Stage Diagram	18
3	Analysis Stage Diagram	19
4	Designing Stage Diagram	20
5	Development Stage Diagram	21
6	Integration & Test Stage Diagram	22
7	Installation & Test Stage Diagram	23
8	Class Diagram	26
9	Use Case Diagram	26
10	Sequence Diagram	27
11	Collaboration Diagram	28
12	Component Diagram	29
13	Deployment Diagram	30
14	Activity Diagram	31
15	Data Flow Diagram	32

LIST OF SCREENSHOTS

Screenshot No.	Particulars	Page No.
1	Dataset	38
2	Application Main Interface	39
3	Dataset Upload Screen	39
4	Dataset Loaded Output	40
5	Data Preprocessing Screen	40
6	Random Forest on Transaction Data	41
7	Random Forest with Time-Frequency Transaction Data	41
8	Comparison Graph of Model Performance	42
9	Money Laundering Prediction Output	42
10	Test Dataset Selection Screen	43

INTRODUCTION

In today's digital world, online advertising has become one of the most powerful business strategies used by companies to promote products and services. Websites, search engines, social media platforms, e-commerce applications, and video streaming services display advertisements continuously to attract users and increase brand visibility. The main challenge in online advertising is to ensure that the right advertisement reaches the right user at the right time. If users are shown irrelevant advertisements, they are less likely to interact with them, which reduces advertiser profit and negatively affects the user's browsing experience. Therefore, predicting user interest in advertisements has become a highly important task in the field of machine learning, data mining, and computational advertising.

The probability that a user clicks on a displayed advertisement is known as the Click-Through Rate (CTR). CTR is one of the most critical performance metrics in digital advertising systems because it directly influences the revenue generated by websites and advertising platforms. In cost-per-click (CPC) and search advertisement models, advertisers pay only when the user clicks the ad. Hence, accurate CTR prediction helps platforms rank advertisements effectively, maximize profit, and improve advertiser trust. A high CTR indicates that the advertisement is relevant and attractive to the user, while a low CTR suggests poor targeting or low user interest. For this reason, companies such as Google, Meta, and Alibaba use advanced CTR prediction systems as the core of their online advertising ecosystem.

This project is based on the development of an Advertisement Click-Through Rate Prediction and Recommendation System, which aims to improve advertisement targeting by analyzing large-scale user and advertisement data. The system mainly works in three stages: data preprocessing, CTR prediction model training, and recommendation generation. In the preprocessing stage, user browsing history, shopping records, advertisement details, and contextual information are collected and cleaned. Since real-world data often contains missing values, redundant attributes, and noise, preprocessing becomes a necessary step before training the model. Useful features are then extracted from user actions such as browsing products, adding items to cart, favoriting products, and purchasing behavior. These features provide valuable insights into user interests and purchasing intentions.

After feature extraction, the processed data is used to train a machine learning model that predicts whether a user is likely to click on a particular advertisement. The project uses the DeepFM algorithm, which combines Factorization Machines (FM) with Deep Neural Networks (DNN). This hybrid model is highly effective because it captures both low-order feature interactions and high-order nonlinear relationships between users and advertisements. Unlike traditional methods such as Logi

stic Regression, which assume feature independence, DeepFM can automatically learn complex patterns from sparse advertising datasets. This makes it highly suitable for real-world advertising environments where millions of users and ads are involved.

Another important aspect of this project is the recommendation system, which works alongside CTR prediction. Once the click probability is calculated, the system identifies the users whose predicted CTR values are above a predefined threshold. These users are then recommended as the most suitable target audience for the advertisement campaign. This helps advertisers improve conversion rates, reduce unnecessary advertisement costs, and achieve better return on investment. At the same time, users receive advertisements that are more relevant to their interests, improving their overall browsing experience.

The significance of this project lies in its practical applications in e-commerce platforms, search engines, social media advertising, and personalized recommendation systems. With the rapid growth of online business, intelligent advertisement prediction systems have become essential for maintaining platform competitiveness. By combining accurate CTR prediction with user recommendation, this project provides a scalable, efficient, and industry-oriented solution that can be widely applied in modern digital marketing systems. Overall, the proposed system not only improves advertisement efficiency but also creates a win-win environment for advertisers, platforms, and end users**.**

LITERATURE SURVEY

"Combining Behavioral and Social Network Data for Online Advertising"

This research focuses on improving advertisement click-through rate prediction by combining user browsing behavior with social network information. The main idea of this approach is that users who are connected through social networks or share similar interests are likely to respond to similar advertisements. By studying both individual behavior and the click behavior of friends or connected users, the model can better understand hidden user preferences. This approach provides a new direction in CTR prediction because it goes beyond traditional user history analysis and includes social influence as an additional feature. The major advantage of this method is that it improves targeting accuracy by utilizing mutual interests and community behavior. However, its limitation is that the performance highly depends on the quality and relevance of social network data, which may not always represent the real interests of users.

"Predicting Clicks: Estimating the Click-Through Rate for New Ads"

This paper mainly addresses the problem of predicting the CTR for newly introduced advertisements that do not yet have enough historical click records. In many real-world advertising systems, new ads are continuously added, and predicting their performance becomes difficult due to the absence of prior data. The researchers solved this issue by using machine learning techniques such as Logistic Regression and by considering features like advertisement terms, bid keywords, ad quality, display URL, and advertisement position. The study found that related search terms and advertisement relevance significantly improve prediction performance. This work is highly useful because it allows advertising platforms to rank new advertisements efficiently even before enough real click data is collected. Its major contribution is reducing the cold-start problem in advertisement systems.

''A Novel Click Model and Its Applications to Online Advertising''

This research proposes an advanced click prediction model based on Bayesian networks. The model studies the behavior of users while they browse advertisements from top to bottom and predicts whether the user will click based on relevance, ranking position, and other contextual factors. Unlike traditional models, this approach captures the sequential examination behavior of users, making the prediction process more realistic. The study also introduces additional features such as local time and user device information, which improve the prediction accuracy further. One of the major strengths of this approach is its strong generalization ability and better performance for complex user queries. However, the model is computationally complex and may require more resources for large-scale industrial implementation.

‘‘Field-weighted Factorization Machines for Click-Through Rate Prediction”

This paper introduces the concept of assigning different weights to interactions between feature fields in CTR prediction. In online advertising datasets, different fields such as user age, browsing history, advertisement type, device, and search keywords do not contribute equally to the prediction result. Some feature interactions are stronger and more important than others. This study proposed Field-weighted Factorization Machines (FwFM) to capture these varying interaction strengths more effectively. The advantage of this approach is that it models real-world data relationships more accurately than traditional Factorization Machines. It improves prediction performance, especially when feature interactions play a significant role. However, the increased complexity of interaction calculations can make the training process slower for very large sparse datasets.

‘‘DeepFM: A Factorization-Machine based Neural Network for CTR Prediction’’

This is one of the most important research works related to our project. DeepFM combines the strengths of **Factorization Machines (FM)** and **Deep Neural Networks (DNN)** into a single hybrid architecture. The FM component captures low-order feature interactions, while the deep neural network component automatically learns high-order nonlinear relationships among features. This reduces the need for manual feature engineering, which is usually a time-consuming task in CTR systems. The model performs exceptionally well on sparse advertising datasets and is widely used in large-scale e-commerce and online advertising platforms. Compared to Logistic Regression, FM, FFM, and PNN, DeepFM provides higher accuracy and better scalability. Due to these advantages, this project adopts **DeepFM as the core algorithm** for building the Advertisement Click-Through Rate Prediction and Recommendation System.

“Product-based Neural Networks for User Response Prediction”

This research introduces **Product-based Neural Networks (PNN)** as an advanced deep learning model for advertisement click prediction. The main purpose of this model is to capture complex interactions between different input features such as user behavior, advertisement category, browsing history, and contextual information. Unlike traditional neural networks, PNN adds a **product layer** before the hidden layers, which explicitly models the relationships between pairs of features. This helps the system understand how combinations of features influence the user’s clicking behavior. For example, a user who frequently searches for shoes may have a higher probability of clicking sports shoe advertisements when combined with related browsing patterns. The advantage of this model is its strong capability to learn nonlinear feature interactions and improve CTR prediction accuracy. However, because the feature interaction space becomes very large, the training complexity and computational cost are also high. This research plays an important role in the

evolution of CTR prediction models and also inspired the DeepFM approach used in our project.

"Logistic Regression for Click-Through Rate Prediction"

This research explains the use of Logistic Regression (LR) as one of the earliest and most widely used algorithms for advertisement CTR prediction. The main advantage of Logistic Regression is its simplicity and ability to produce probability outputs between 0 and 1, which perfectly matches the CTR prediction requirement. It works well for large-scale industrial datasets because it is easy to train and can be updated frequently with new user click data. Many early advertising platforms used LR because of its efficiency and fast response time. However, the major limitation is that it assumes independence among input features and fails to capture complex interactions between user behavior and advertisement attributes. This reduces its performance when the dataset becomes highly sparse and nonlinear. Even though it is simple, LR forms the theoretical foundation for many advanced CTR prediction models.

"Factorization Machines for Sparse Advertisement Data"

This paper proposes **Factorization-supported Neural Networks (FNN)**, where the neural network is initialized using embeddings learned from Factorization Machines. The major objective of this method is to combine the advantages of FM feature embeddings with the nonlinear learning ability of neural networks. By starting with FM-based embeddings, the network can learn meaningful feature representations faster and achieve better convergence during training. This is highly useful in CTR prediction tasks where sparse user and ad data create difficulties for standard deep neural networks. The advantage of FNN is its ability to learn higher-order feature interactions while still benefiting from FM's sparse data handling. However, it requires a pretraining step, which increases implementation complexity. This work strongly influenced later hybrid models like DeepFM

"Wide & Deep Learning for Recommender Systems"

This research introduces the Wide & Deep Learning (WDL) model, which combines linear models with deep neural networks for recommendation and CTR prediction tasks. The wide component memorizes simple co-occurrence relationships between features, while the deep component generalizes complex feature interactions. In advertisement systems, this allows the model to learn both frequent click patterns and hidden nonlinear user preferences. WDL became highly influential because it successfully balances memorization and generalization. Many industrial systems adopted this approach due to its strong real-world performance. However, the wide part still depends on manual feature engineering, which increases the workload. This

drawback was later reduced in DeepFM by replacing the wide component with Factorization Machines.

"User Behavior Analysis in Online Advertisement Systems"

User behavior analysis is one of the most important foundations of modern click-through rate prediction systems. Every action performed by a user on an online platform gives valuable information about their interests and preferences. Activities such as searching for products, viewing multiple items in the same category, adding products to the cart, saving wish list items, and completing purchases all help in understanding user intent. By continuously tracking these behavioral patterns, the system can identify what kind of advertisements are most likely to attract a user. For example, if a user frequently browses smartphones and accessories, the probability of clicking electronics-related advertisements becomes much higher than unrelated categories such as furniture or clothing. This behavioral understanding makes advertisement delivery more personalized and effective. It also helps reduce the display of irrelevant ads, which improves user satisfaction and increases the overall performance of digital advertising platforms.

"Feature Interaction Methods in CTR Prediction"

Feature interaction plays a major role in determining whether a user clicks on a displayed advertisement. In real-world online advertising systems, user-related features and advertisement-related features do not work independently. Instead, the interaction between these features decides the final click probability. For example, the combination of a user's age, shopping interest, browsing device, time of access, and advertisement category may together influence the click decision. Learning these hidden relationships manually is difficult because the number of possible combinations becomes extremely large. Advanced CTR prediction methods solve this problem by automatically learning feature interactions using embedding layers and interaction networks. These techniques are highly useful in sparse datasets where many categorical variables are involved. Better interaction learning directly improves CTR accuracy because it captures real user-advertisement relationships more effectively. This concept is one of the major reasons why hybrid models like DeepFM perform better than simple linear approaches.

"Deep Learning Approaches for Advertisement Recommendation"

Deep learning has significantly transformed online advertisement recommendation systems by improving their ability to learn complex user behavior patterns from large-scale data. Traditional machine learning methods often depend heavily on manual feature engineering, which requires domain knowledge and significant effort. In contrast, deep learning models automatically learn useful hidden representations from raw user actions and advertisement features. These models can capture nonlinear relationships between browsing history, search patterns, contextual information, and ad categories. This makes them highly effective in environments where user behavior continuously changes over time. Another major advantage is scalability, as deep learning architectures can efficiently process millions of user interactions and advertisement records. In modern e-commerce and social media platforms, these approaches help improve personalization, increase advertisement relevance, and maximize click probability. Because of their strong performance in handling sparse and high-dimensional data, deep learning models form the backbone of advanced CTR systems.

"Personalized Recommendation using User Preference Modeling"

Personalized recommendation systems aim to provide users with advertisements that match their unique interests and preferences. Instead of showing the same advertisement to all users, the system studies the browsing history, product categories visited, frequently searched keywords, previous purchases, and click patterns of each individual user. From this information, a preference model is created to understand the products or services the user is most likely interested in. For example, a user who often searches for sports shoes, gym equipment, and fitness accessories is more likely to respond positively to sports-related advertisements. This personalization increases the chances of advertisement clicks and improves user engagement. It also creates a better user experience because advertisements become more useful rather than distracting. In online business platforms, personalized recommendation systems are extremely valuable because they help advertisers target the right audience and improve conversion rates while reducing advertisement waste.

"Personalized Recommendation using User Preference Modeling"

Personalized recommendation systems aim to provide users with advertisements that match their unique interests and preferences. Instead of showing the same advertisement to all users, the system studies the browsing history, product categories visited, frequently searched keywords, previous purchases, and click patterns of each individual user. From this information, a preference model is created to understand the products or services the user is most likely interested in. For example, a user who often searches for sports shoes, gym equipment, and fitness accessories is more likely to respond positively to sports-related.

3. SYSTEM ANALYSIS

System analysis is the process of studying the overall working of the project before implementation. In this project, the system is analyzed based on user browsing behavior, advertisement details, and click history to predict the click-through rate of online advertisements. The analysis helps in understanding data flow, feature relationships, prediction accuracy, and recommendation performance. It also helps in selecting the most suitable algorithm for efficient advertisement targeting and user recommendation.

3.1 Existing System

The existing system in online advertisement platforms mainly uses traditional machine learning methods such as Logistic Regression, basic recommendation models, and manual feature engineering techniques for click-through rate prediction. These systems depend mostly on historical click records and simple user-advertisement relationships to estimate the probability of clicks. While these approaches are easy to implement and computationally efficient, they are not highly effective for handling large-scale sparse datasets generated in real-time advertising environments.One of the major drawbacks of the existing system is that it cannot efficiently capture complex feature interactions between user behavior, advertisement content, browsing context, and device information. Since online user data is highly dynamic and nonlinear, traditional methods often fail to model hidden relationships properly. This leads to lower prediction accuracy and poor advertisement targeting.Another limitation is the heavy dependency on manual feature engineering, where domain experts need to create useful interaction features explicitly. This process requires more time, effort, and expert knowledge. It also becomes difficult to scale when the number of users, advertisements, and contextual attributes increases rapidly.Because of these limitations, the existing system often shows lower CTR prediction performance, reduced personalization, and less effective advertisement recommendations**,** which may decrease user engagement and advertiser revenue.

3.1.2 Disadvantages of Existing System

Despite advancements, the existing system has several limitations:

Low Prediction Accuracy:

The existing systems are not highly accurate in predicting whether a user will click on an advertisement, especially when user behavior is complex and dynamic in nature.

Poor Handling of Feature Interactions:

Traditional models fail to properly capture the relationship between user features and advertisement features, leading to weak prediction performance.

High False Predictions:

The system often predicts incorrect click probabilities, resulting in either showing irrelevant advertisements or missing potential interested users.

Limited Personalization:

Existing systems do not effectively understand individual user preferences, which leads to the same advertisements being shown to many unrelated users.

Scalability Issues:

When dealing with large-scale datasets containing millions of users and advertisements, the system performance decreases and becomes less efficient.

Heavy Manual Feature Engineering:

The existing system requires manual selection and creation of features, which is time-consuming and requires domain expertise.

3.2 PROPOSED SYSTEM

The proposed system is designed to improve the accuracy of Click-Through Rate (CTR) prediction and provide an efficient advertisement recommendation solution using machine learning techniques. Unlike traditional systems that rely on manual rules and simple statistical methods, this system uses advanced data-driven learning to understand user behavior and advertisement relationships more effectively.In the proposed system, user behavior data such as browsing history, search patterns, click records, and advertisement interaction details are collected and preprocessed. This cleaned data is then used to extract meaningful features that represent user interests and advertisement characteristics in a structured format. These features help the system to better understand how users interact with different types of advertisements.The system uses the DeepFM (Deep Factorization Machine) algorithm as the core model for prediction. DeepFM combines the advantages of Factorization Machines and Deep Neural Networks to capture both low-order and high-order feature interactions automatically. This eliminates the need for heavy manual feature engineering and improves the overall prediction performance. It is especially effective in handling large-scale and sparse datasets commonly found in online advertising systems.Based on the predicted click-through rate values, the system identifies users who are more likely to click on a particular advertisement. These users are then used for recommendation, ensuring that advertisements are shown only to the most relevant audience. This improves targeting efficiency and increases the chances of conversion.Overall, the proposed system provides a more intelligent, scalable, and accurate solution for advertisement prediction and recommendation. It reduces irrelevant advertisement display, improves user engagement, and helps advertisers achieve better performance and higher returns on investment.

3.2.1 Advantages of Proposed System

The proposed system offers several advantages over the existing system:

1. **Improved Prediction Accuracy:**

   The system provides better accuracy in predicting whether a user will click on an advertisement by using advanced machine learning techniques.

2. **Better Feature Learning:**

   It automatically learns important relationships between user behavior and advertisement features, improving the quality of predictions.

3. **Improved Personalization:**

T Advertisements are shown based on individual user interests, leading to more relevant and effective ad targeting.

4. **Increased Advertiser Profit:**

By targeting the right users, the system improves click rates and helps advertisers achieve higher returns on investment.

5. **Scalable and Flexible System:**

The model can be extended and adapted for real-time advertisement prediction in modern digital platforms.

3.2.2 Limitations of Advanced System

Although the proposed system improves detection, it still has some limitations:

Data Dependency:

The performance of the system highly depends on the quality, quantity, and relevance of user behavior and advertisement data ..

Computational Complexity:

The DeepFM model and feature processing steps may require higher computational resources and longer training time for large datasets.

Need for Proper Training:

The model requires careful training, tuning, and parameter optimization to achieve the best prediction performance.

Data Privacy Concerns:

Since the system uses user behavior data, maintaining privacy and secure handling of data becomes an important requirement.

4. SYSTEM STUDY

System study is an important phase in system development that evaluates whether the proposed system is practical, efficient, and beneficial. It focuses on analyzing the feasibility of the system from different perspectives such as economic, operational, and technical aspects. The main objective of this study is to ensure that the proposed Advertisement Click-Through Rate (CTR) Prediction and Recommendation System can be successfully implemented in real-world online advertising environments.

In this project, the system study helps in understanding whether the proposed CTR prediction model can effectively analyze user behavior, advertisement data, and browsing patterns to improve advertisement targeting. A detailed feasibility study is carried out to evaluate the practicality and performance of the system.

4.1 FEASIBILITY STUDY

Feasibility study is conducted to evaluate whether the proposed system is practical and beneficial before implementation. It helps in identifying whether the system can be developed within available resources and whether it will provide expected improvements. The feasibility study of this project is divided into three main categories:

- Economic Feasibility
- Operational Feasibility
- Technical Feasibility

4.1.1 ECONOMIC FEASIBILITY

Economic feasibility evaluates whether the proposed system is cost-effective and whether the benefits justify the development cost.

In this project, the Advertisement Click-Through Rate Prediction System does not require expensive hardware or software resources. It can be developed using commonly available tools such as Python, machine learning libraries, and standard computing systems. Therefore, the overall development cost is low.

The major benefit of the system is its ability to improve advertisement targeting and increase click-through rates, which directly leads to higher revenue for advertisers and advertising platforms. By showing relevant advertisements to the right users, the system reduces wasted advertisement costs and improves marketing efficiency.

Additionally, the system reduces the need for manual analysis of advertisement performance, thereby saving time and operational costs. Since it improves accuracy and profitability while maintaining low development cost, the system is economically feasible.

4.1.2 OPERATIONAL FEASIBILITY

Operational feasibility determines whether the system can function effectively in real-world advertising environments and whether it will be accepted by users.The proposed system is designed to be easily integrated into existing online advertising platforms such as e-commerce websites, search engines, and recommendation systems. It does not require major changes in current business workflows, making it easy to adopt.The system provides clear outputs such as predicted click probability and recommended target users, which can be easily understood by advertisers and platform administrators. This improves usability and decision-making.The system also reduces manual effort by automating CTR prediction and advertisement targeting processes. This helps improve efficiency and reduces human errors in ad selection. Since the system improves performance and is easy to operate, it is operationally feasible.

4.1.3 TECHNICAL FEASIBILITY

Technical feasibility evaluates whether the system can be developed and implemented using available technologies.The proposed system is technically feasible as it uses widely accepted machine learning and deep learning techniques. The system is developed using Python and libraries such as Pandas, NumPy, and Scikit-learn for data processing and model building.The DeepFM algorithm is used as the core prediction model because it effectively captures both low-level and high-level feature interactions. This makes it highly suitable for large-scale and sparse advertisement datasets.

The system processes user behavior data such as browsing history, advertisement clicks, and contextual features to train the model. It can be deployed as a web-based application or integrated into existing advertisement platforms for real-time prediction.

Since the system is built using scalable and well-supported technologies, and can handle large volumes of data efficiently, it is technically feasible.

5. HARDWARE & SOFTWARE REQUIREMENTS

The hardware and software requirements define the essential components needed for the development and execution of the proposed Advertisement Click-Through Rate Prediction and Recommendation System. These requirements ensure that the system runs efficiently and provides accurate prediction results. In this project, the system requirements are kept minimal so that it can be implemented easily without high cost.

5.1 HARDWARE REQUIREMENTS

Hardware requirements refer to the physical components necessary to run the system. The proposed system does not require high-end hardware and can be executed on standard computing devices. The following are the minimum hardware requirements:

- Processor: Intel i3 (Minimum)
- Speed: 1.1 GHz or higher
- RAM: 4 GB (Minimum)
- Hard Disk: 500 GB
- Keyboard: Standard Windows Keyboard
- Mouse: Two or Three Button Mouse
- Monitor: SVGA Monitor

5.1.1Explanation of Hardware Requirements

The processor is responsible for executing machine learning algorithms and processing user behavior and advertisement data. An Intel i3 processor is sufficient for basic model training and prediction tasks, although higher processors like i5 or i7 can improve performance and reduce execution time.

A minimum of 4 GB RAM is required to handle data processing and machine learning operations efficiently. Since the system deals with user interaction data and advertisement datasets, sufficient memory is necessary for smooth execution.

The hard disk is used to store datasets, trained models, program files, and output results. A storage capacity of 500 GB is sufficient for moderate-scale advertisement datasets.

Input devices such as keyboard and mouse are used for interacting with the system, while the monitor displays results such as predicted CTR values and recommendation outputs.

Overall, the hardware requirements are simple and cost-effective, making the system easy to implement on most basic computing systems.

5.2 SOFTWARE REQUIREMENTS

Software requirements refer to the programs and tools required to develop and run the system. The proposed system is developed using widely available and open-source software technologies.

- **Operating System**: Windows 10 (Minimum)
- **Programming Language**: Python

Additional Tools and Libraries

In addition to the basic software, the following tools and libraries are used:

- **Python Libraries**:
  - Pandas (for data handling)
  - NumPy (for numerical computations)
  - Scikit-learn (for machine learning models)
  - Matplotlib / Seaborn (for data visualization)
- **Development Environment**:
  - Jupyter Notebook / VS Code / PyCharm
- **Database (Optional)**:
  - MySQL / SQLite

5.2.1 Explanation of Software Requirements

The system is developed using Python, which is widely used for data analysis and machine learning applications. Python provides powerful libraries that simplify data processing and model development.

The operating system used is Windows 10, which supports all the required tools and provides a user-friendly interface. The system can also be adapted to run on other operating systems such as Linux or macOS.

Libraries such as Pandas and NumPy are used for handling and processing transaction data efficiently. Scikit-learn is used to implement the Random Forest algorithm for classifying transactions. Visualization tools like Matplotlib help in analyzing patterns and presenting results.

The development environment such as Jupyter Notebook or VS Code provides a platform for writing, testing, and debugging the code. A database system may be used to store transaction records and manage data efficiently.

6. Software Requirement Specification

6.1 Overall Description

The Software Requirement Specification (SRS) is a detailed description of the system to be developed. It defines the functional and non-functional requirements of the system and acts as a reference document for developers, designers, and stakeholders. The SRS provides a clear understanding of the system's behavior, features, and constraints, ensuring that the final product meets user expectations.

In this project, the SRS defines the requirements for a suspicious activity detection system based on time-frequency analysis and machine learning. It describes how the system will process transaction data, detect abnormal patterns, and generate alerts for suspicious activities.

6.2 Functional Requirements

Functional requirements describe what the system should do. These requirements define the core operations and features of the system.

- **User Data Input:**
  The system should accept user behavior data such as browsing history, search activity, and advertisement interaction details.
- **Data Preprocessing:**
  The system should clean and preprocess raw data by handling missing values, duplicates, and irrelevant information.
- **Feature Extraction:**
  The system should extract meaningful features from user behavior and advertisement data for model training.
- **CTR Prediction:**
  The system should predict the probability of a user clicking on a specific advertisement using the DeepFM model.
- **Recommendation Generation:**
  The system should recommend advertisements to users based on predicted CTR values.
- **Data Storage:**
  The system should store processed data, predictions, and results for future analysis

6.3 Non-Functional Requirements

- **Performance:**

  The system should provide fast and accurate predictions even for large datasets.

- **Security:**

  User data should be securely handled to protect privacy and prevent unauthorized access.

- **Scalability:**

  The system should support large-scale advertising datasets and increasing user traffic.

- **Reliability:**

  The system should consistently provide accurate and stable prediction results.

- **Usability:**

  The system should be easy to use for advertisers and system administrators.

6.4 Types of Requirements

In system development, requirements are generally categorized into three types:

Business Requirements

The main business goal is to improve advertisement targeting, increase click-through rates, and maximize advertiser revenue.

Product Requirements

These describe the features and functions of the system.
For this project, the system must detect suspicious activities, analyze transaction patterns, and generate alerts.

Process Requirements

The system must analyze user behavior, predict CTR values, and recommend relevant advertisements to users.

6.5 Role of System Analyst

A system analyst plays a key role in understanding user requirements and translating them into system The system analyst plays an important role in understanding requirements and designing system functionality.In this project, the system analyst is responsible for:

• Understanding user behavior and advertisement system needs

• Designing data flow and system architecture

• Selecting suitable machine learning models (DeepFM)

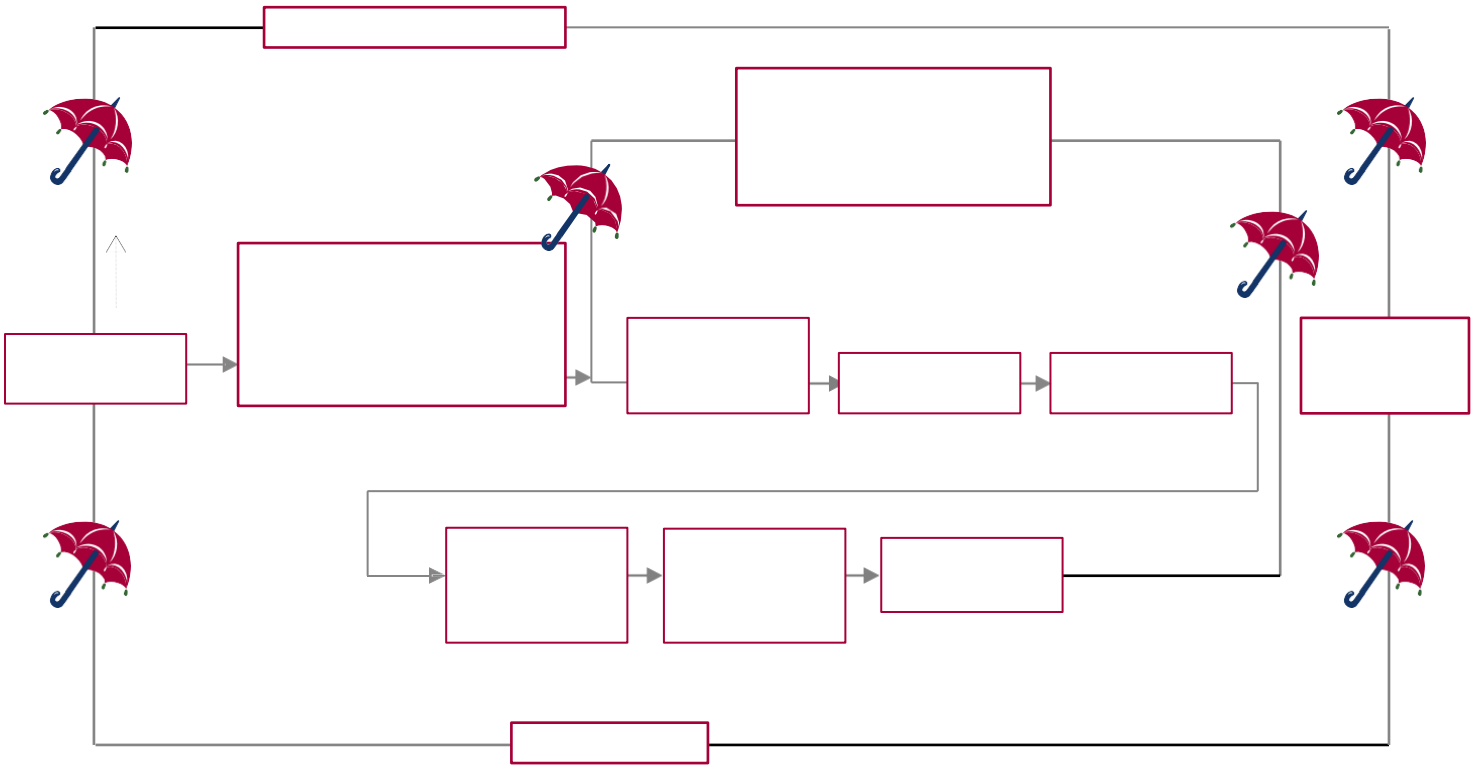
• Ensuring system accuracy and performance

6.6 Feasibility Consideration

Before implementation, feasibility is analyzed to ensure the system is practical and effective. The proposed system is feasible because it uses existing machine learning technologies, requires low implementation cost, and provides high benefits in advertisement targeting and user personalization.

7. ARCHITECTURE

7.1 SDLC (Umbrella Model)



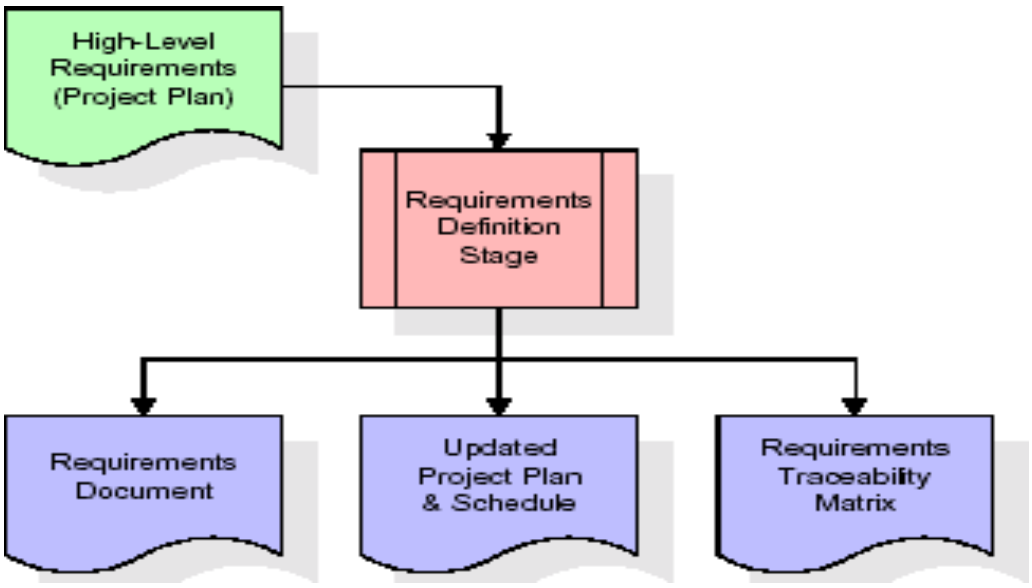
SDLC is nothing but Software Development Life Cycle. It is a standard which is used by software industry to develop good software.

Stages in SDLC

- Requirement Gathering
- Analysis
- Designing
- Coding
- Testing
- Maintenance

Requirements Gathering Stage

The requirements gathering process takes as its input the goals identified in the high-level requirements section of the project plan. Each goal will be refined into a set of one or more requirements. These requirements define the major functions of the intended application, define operational data areas and reference data areas, and define the initial data entities. Major functions include critical processes to be managed, as well as mission critical inputs, outputs and reports. A user class hierarchy is developed and associated with these major functions, data areas, and data entities. Each of these definitions is termed a Requirement. Requirements are identified by unique requirement identifiers and, at minimum, contain a requirement title and textual description.



These requirements are fully described in the primary deliverables for this stage: the Requirements Document and the Requirements Traceability Matrix (RTM). The requirements document contains complete descriptions of each requirement, including diagrams and references to external documents as necessary. Note that detailed listings of database tables and fields are *not* included in the requirements document.

The title of each requirement is also placed into the first version of the RTM, along with the title of each goal from the project plan. The purpose of the RTM is to show that the product components developed during each stage of the software development lifecycle are formally connected to the components developed in prior stages.

In the requirements stage, the RTM consists of a list of high-level requirements, or goals, by title, with a listing of associated requirements for each goal, listed by requirement title. In this hierarchical listing, the RTM shows that each requirement developed during this stage is formally linked to a specific product goal. In this format, each requirement can be traced to a specific product goal, hence the term requirements traceability.

The outputs of the requirements definition stage include the requirements document, the RTM, and an updated project plan.

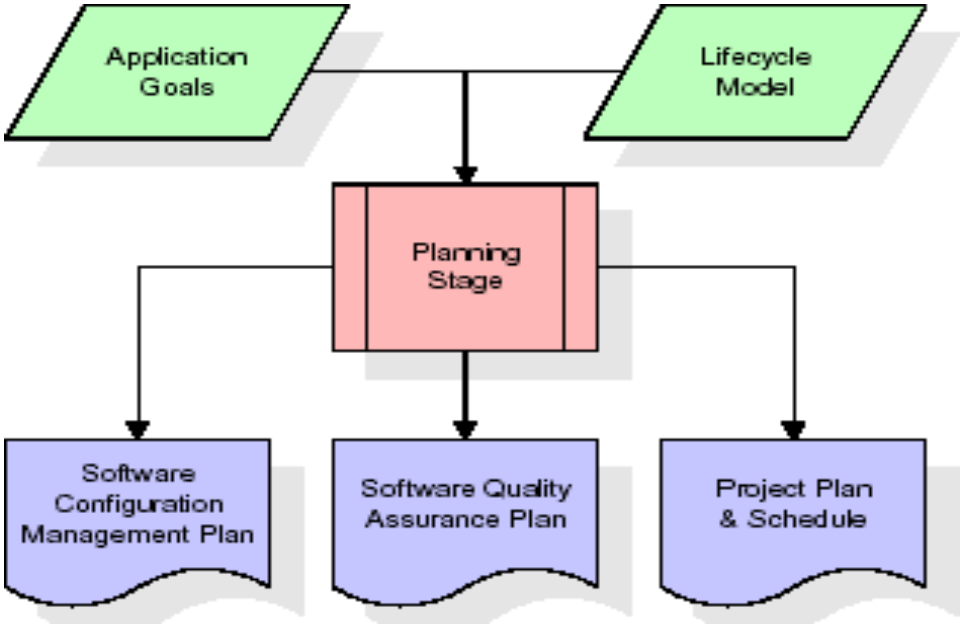
♦ Feasibility study is all about identification of problems in a project.

♦ No. of staff required to handle a project is represented as Team Formation, in this case only modules are individual tasks will be assigned to employees who are working for that project.

♦ Project Specifications are all about representing of various possible inputs submitting to the server and corresponding outputs along with reports maintained by administrator.

Analysis Stage

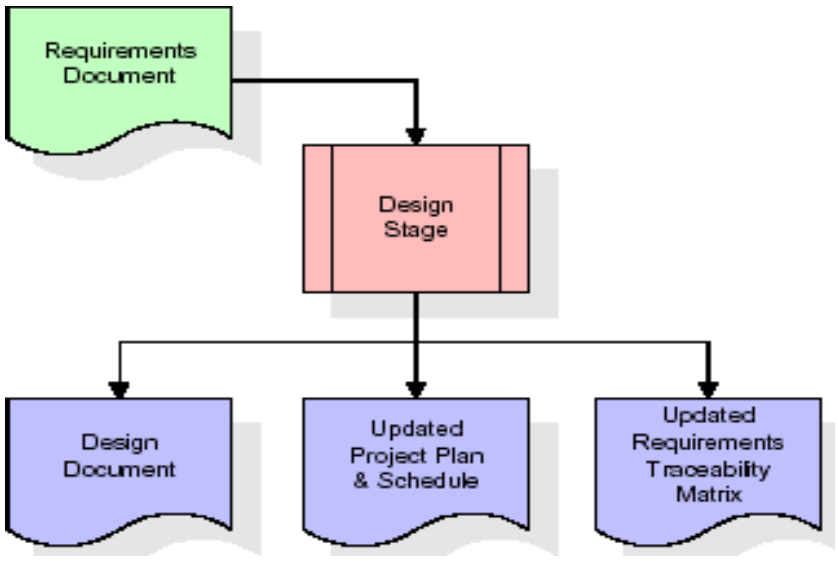
The planning stage establishes a bird's eye view of the intended software product, and uses this to establish the basic project structure, evaluate feasibility and risks associated with the project, and describe appropriate management and technical approaches.



The most critical section of the project plan is a listing of high-level product requirements, also referred to as goals. All of the software product requirements to be developed during the requirements definition stage flow from one or more of these goals. The minimum information for each goal consists of a title and textual description, although additional information and references to external documents may be included. The outputs of the project planning stage are the configuration management plan, the quality assurance plan, and the project plan and schedule, with a detailed listing of scheduled activities for the upcoming Requirements stage, and high level estimates of effort for the out stages.

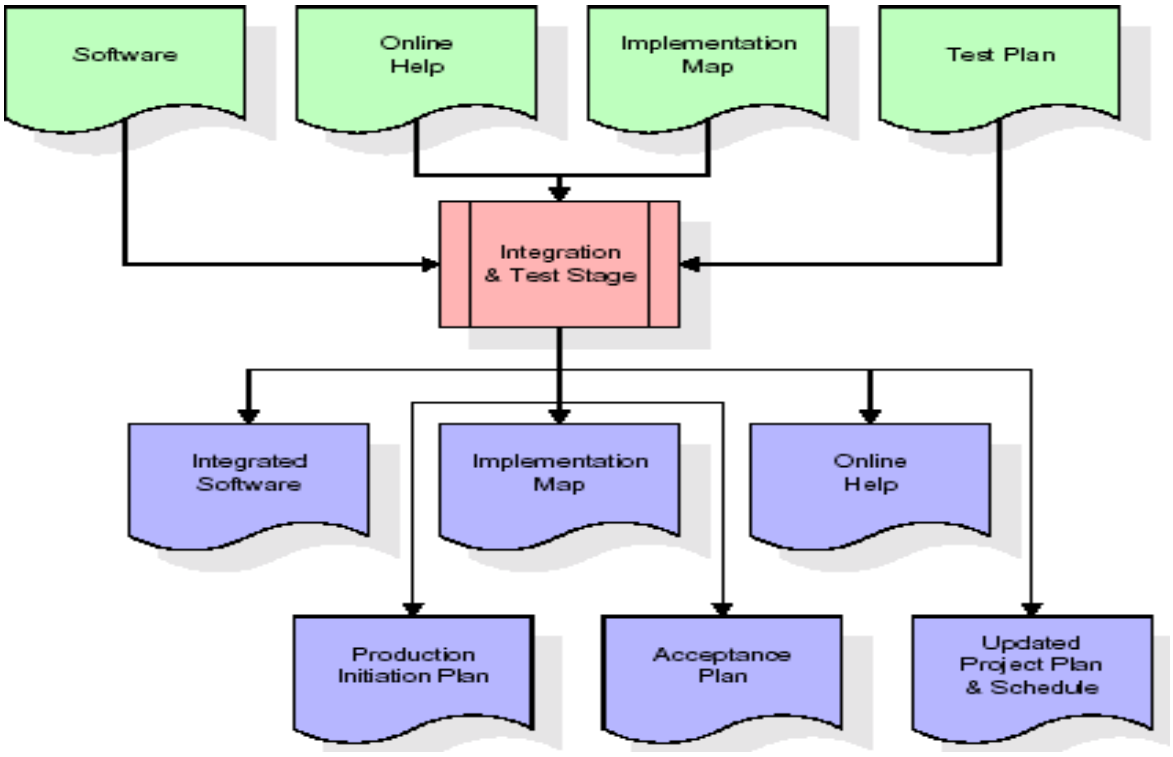
Designing Stage

The design stage takes as its initial input the requirements identified in the approved requirements document. For each requirement, a set of one or more design elements will be produced as a result of interviews, workshops, and/or prototype efforts. Design elements describe the desired software features in detail, and generally include functional hierarchy diagrams, screen layout diagrams, tables of business rules, business process diagrams, pseudo code, and a complete entity-relationship diagram with a full data dictionary. These design elements are intended to describe the software in sufficient detail that skilled programmers may develop the software with minimal additional input.



When the design document is finalized and accepted, the RTM is updated to show that each design element is formally associated with a specific requirement. The outputs of the design stage are the design document, an updated RTM, and an updated project plan.

Development (Coding) Stage



The development stage takes as its primary input the design elements described in the approved design document. For each design element, a set of one or more software artifacts will be produced. Software artifacts include but are not limited to menus, dialogs, and data management forms, data reporting formats, and specialized procedures and functions. Appropriate test cases will be developed for each set of functionally related software artifacts, and an online help system will be developed to guide users in their interactions with the software.

The RTM will be updated to show that each developed artifact is linked to a specific design element, and that each developed artifact has one or more corresponding test case items. At this point, the RTM is in its final configuration. The outputs of the development stage include a fully functional set of software that satisfies the requirements and design elements previously documented, an online help system that describes the operation of the software, an implementation map that identifies the primary code entry points for all major system functions, a test plan that describes the test cases to be used to validate the correctness and completeness of the software, an updated RTM, and an updated project plan.

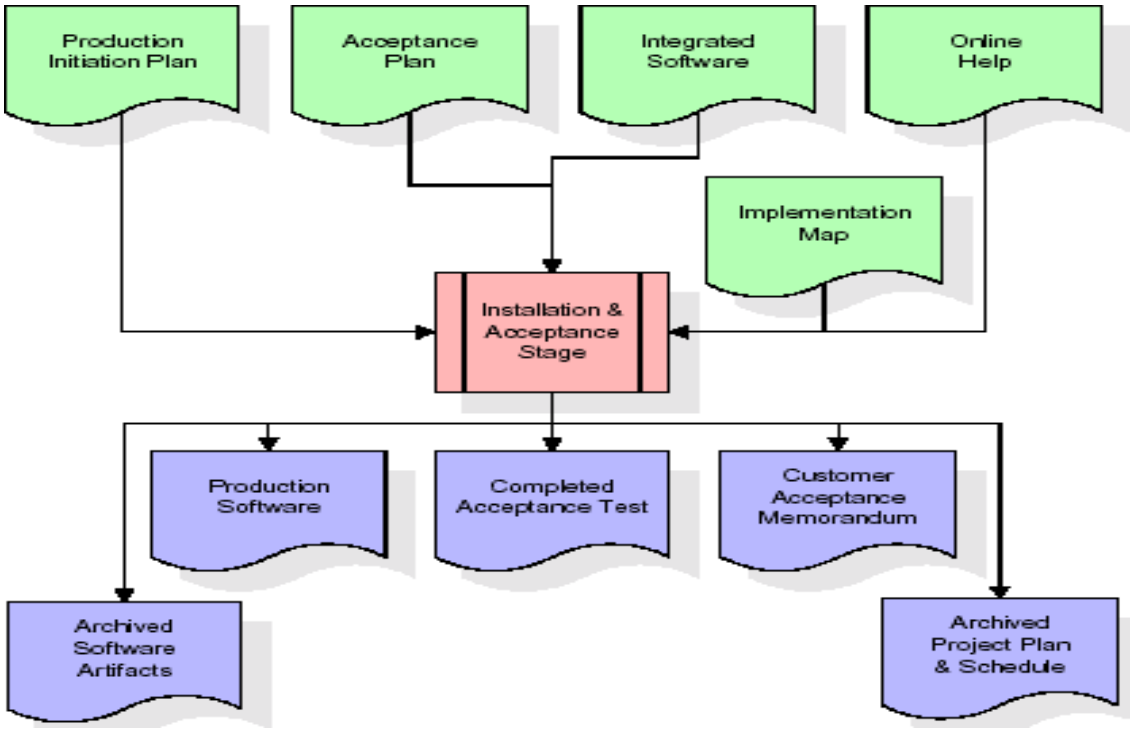
Integration & Test Stage

During the integration and test stage, the software artifacts, online help, and test data are migrated from the development environment to a separate test environment. At this point, all test cases are run to verify the correctness and completeness of the software. Successful execution of the test suite confirms a robust and complete migration capability. During this stage, reference data is finalized for production use and production users are identified and linked to their appropriate roles. The final reference data (or links to reference data source files) and production user list are compiled into the Production Initiation Plan.The outputs of the integration and test stage include an integrated set of software, an online help system, an implementation map, a production initiation plan that describes reference data and production users, an acceptance plan which contains the final suite of test cases, and an updated project plan.

Installation & Acceptance Test

During the installation and acceptance stage, the software artifacts, online help, and initial production data are loa ded onto the production server. At this point, all test cases are run to verify the correctness and completeness of the software. Successful execution of the test suite is a prerequisite to acceptance of the software by the customer.

After customer personnel have verified that the initial production data load is correct and the test suite has been executed with satisfactory results, the customer formally accepts the delivery of the software.



The primary outputs of the installation and acceptance stage include a production application, a completed acceptance test suite, and a memorandum of customer acceptance of the software. Finally, the PDR enters the last of the actual labor data into the project schedule and locks the project as a permanent project record. At this point the PDR "locks" the project by archiving all software items, the implementation map, the source code, and the documentation for future reference.

Maintenance

Outer rectangle represents maintenance of a project, Maintenance team will start with requirement study, understanding of documentation later employees will be assigned work and they will undergo training on that particular assigned category. For this life cycle there is no end, it will be continued so on like an umbrella (no ending point to umbrella sticks).

8. MODULES

The system is divided into multiple modules to simplify the implementation and improve the overall efficiency of the Advertisement Click-Through Rate (CTR) Prediction and Recommendation System. Each module is designed to perform a specific function in the process of analyzing user data, predicting click probabilities, and recommending suitable advertisements. These modules work together in a sequential manner to ensure smooth data flow and accurate prediction results. The modular design also makes the system easy to understand, maintain, and scale for future enhancements.

8.1 Upload Advertisement Dataset

This module is responsible for uploading the dataset into the system. The dataset includes user-related information, advertisement details, browsing history, and click data. Once the dataset is uploaded, the system reads and stores it for further processing. It also provides an initial overview of the dataset, such as the number of records, features, and basic statistics. This module acts as the entry point of the system where raw data is collected and prepared for further analysis.

8.2 Data Preprocessing

Feature extraction is an important module where meaningful information is derived from raw data. In this system, user behavior such as browsing history, product views, cart additions, and purchase activities are analyzed to extract useful features.

Advertisement-related attributes such as category, keywords, and display characteristics are also considered. These extracted features help in identifying patterns between users and advertisements, which play a key role in predicting the click-through rate.

8.3 Feature Extraction

Feature extraction is an important module where meaningful information is derived from raw data. In this system, user behavior such as browsing history, product views, cart additions, and purchase activities are analyzed to extract useful features.Advertisement-related attributes such as category, keywords, and display characteristics are also considered. These extracted features help in identifying patterns between users and advertisements, which play a key role in predicting the click-through rate.

8.4 CTR Prediction using DeepFM Model

In this module, the processed and feature-engineered dataset is used to train the prediction model. The system uses the DeepFM algorithm, which combines Factorization Machines and Deep Neural Networks. The model learns both low-level and high-level feature interactions automatically, making it highly suitable for sparse and complex advertisement data. After training, the model predicts the probability of a user clicking on a particular advertisement. This module forms the core of the system as it generates accurate CTR predictions.

8.5 Recommendation System

This module works based on the predicted CTR values generated by the model. The system identifies users whose predicted click probability is higher than a predefined threshold.

These users are recommended as the target audience for specific advertisements. This helps advertisers focus on the right users, improving conversion rates and reducing unnecessary advertisement costs. It also enhances user experience by showing relevant advertisements.

8.6 Performance Evaluation and Visualization

This module is used to evaluate the performance of the prediction model. The system calculates metrics such as accuracy, precision, recall, and F1-score to measure the effectiveness of the model.

Additionally, graphical representations such as comparison graphs and performance charts are generated to visualize the results. This helps in analyzing how well the model performs and demonstrates the improvements achieved by the system.

8.7 Predict CTR for New Data

This module is used for real-time prediction. Users can provide new input data, and the trained model will predict whether a user is likely to click on an advertisement.

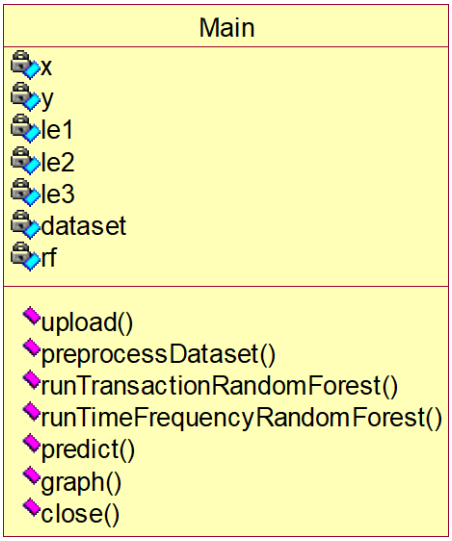
The output is presented in a simple and understandable format, showing the probability of a click. This module makes the system practical and usable in real-world applications where continuous prediction is required.

9. DIAGRAMS

9.1 CLASS DIAGRAM

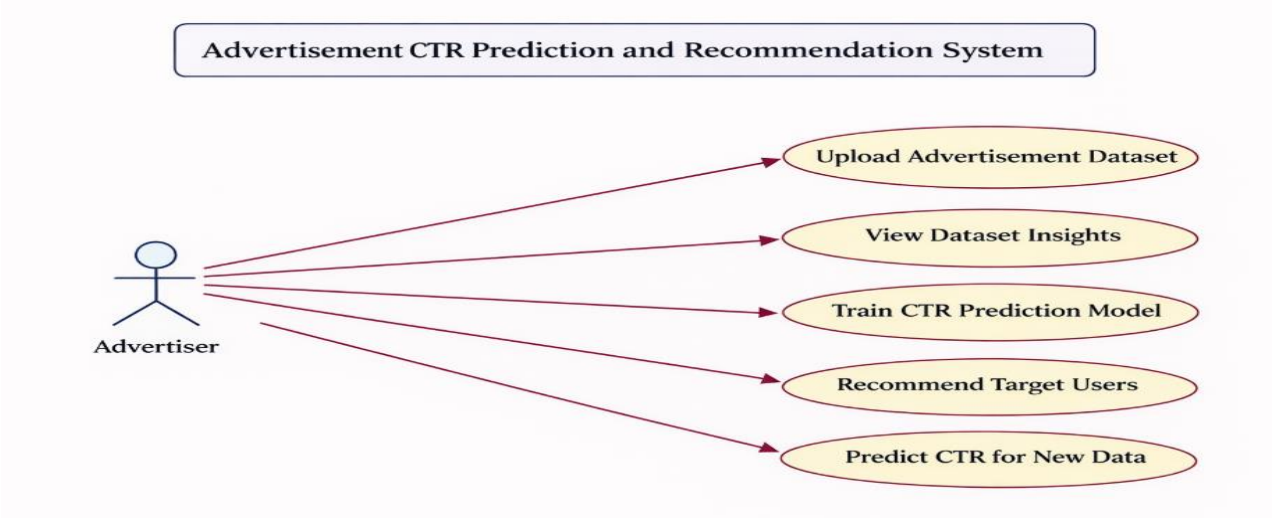
The class diagram is the main building block of object oriented modeling. It is used both for general conceptual modeling of the systematic of the application, and for detailed modeling translating the models into programming code. Class diagrams can also be used for data modeling. The classes in a class diagram represent both the main objects, interactions in the application and the classes to be programmed. In the diagram, classes are represented with boxes which contain three parts:

- The upper part holds the name of the class
- The middle part contains the attributes of the class
- The bottom part gives the methods or operations the class can take or undertake



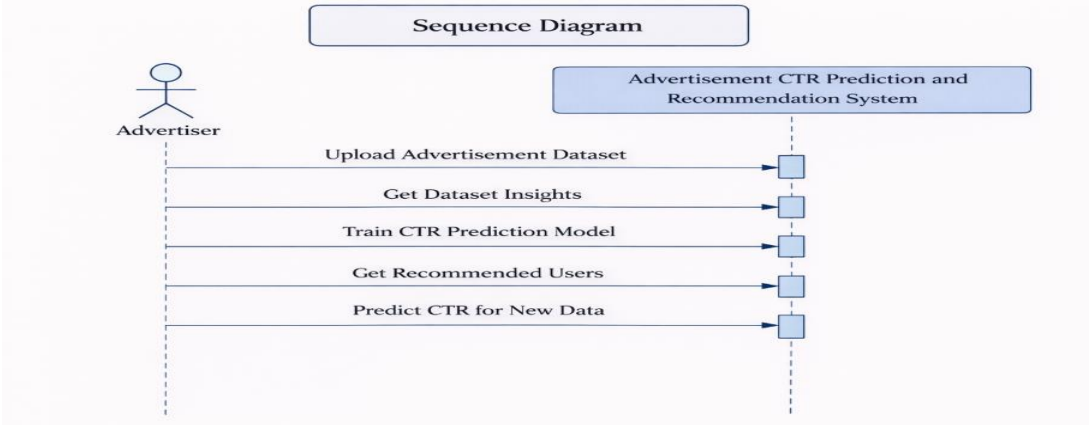
9.2 USECASE DIAGRAM

A **use case diagram** at its simplest is a representation of a user's interaction with the system and depicting the specifications of a use case. A use case diagram can portray the different types of users of a system and the various ways that they interact with the system. This type of diagram is typically used in conjunction with the textual use case and will often be accompanied by other types of diagrams as well.



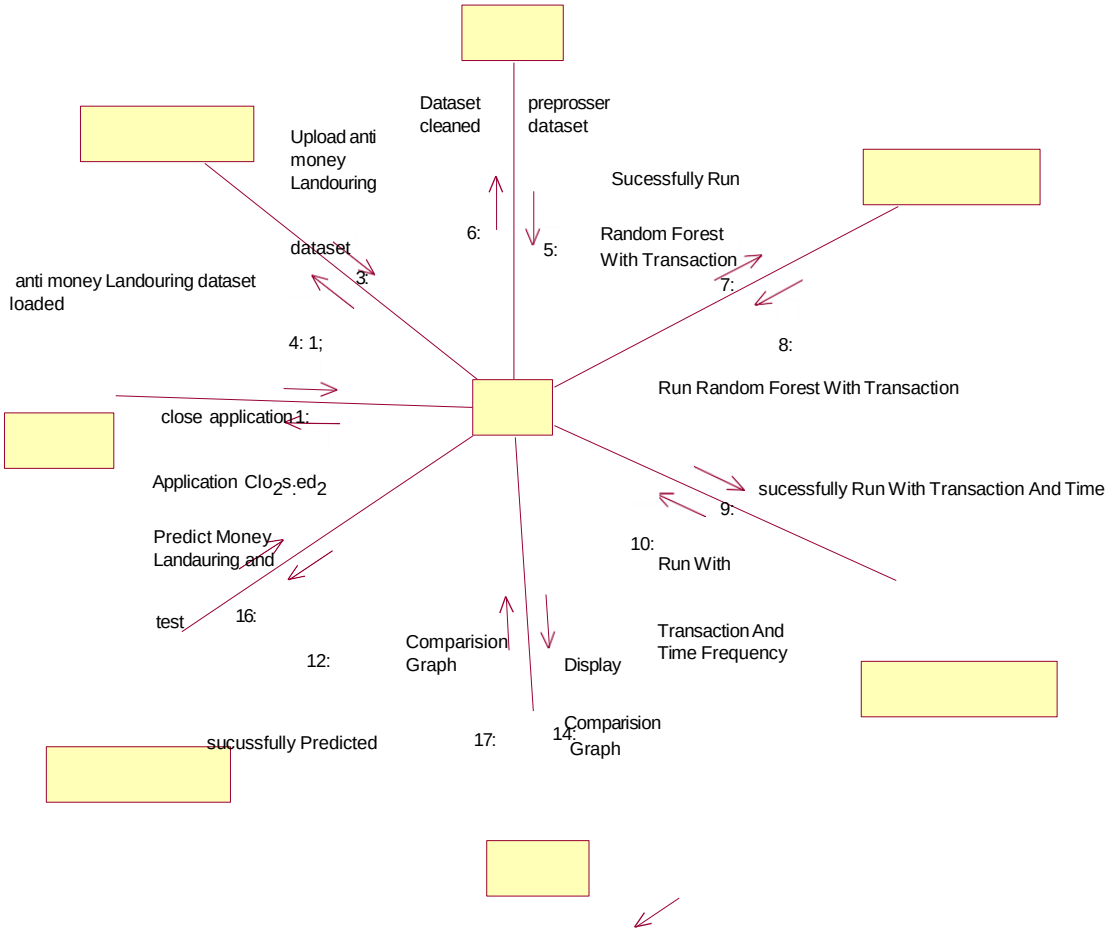
9.3 SEQUENCE DIAGRAM

A **sequence diagram** is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development. Sequence diagrams are sometimes called **event diagrams**, **event scenarios**, and timing diagrams.



9.4 COLLABORATION DIAGRAM

A collaboration diagram describes interactions among objects in terms of sequenced messages. Collaboration diagrams represent a combination of information taken from class, sequence, and use case diagrams describing both the static structure and dynamic behaviour of a system.



9.5 COMPONENT DIAGRAM

In the Unified Modelling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems.

Components are wired together by using an assembly connector to connect the required interface of one component with the provided interface of another component. This illustrates the service consumer - service provider relationship between the two components.

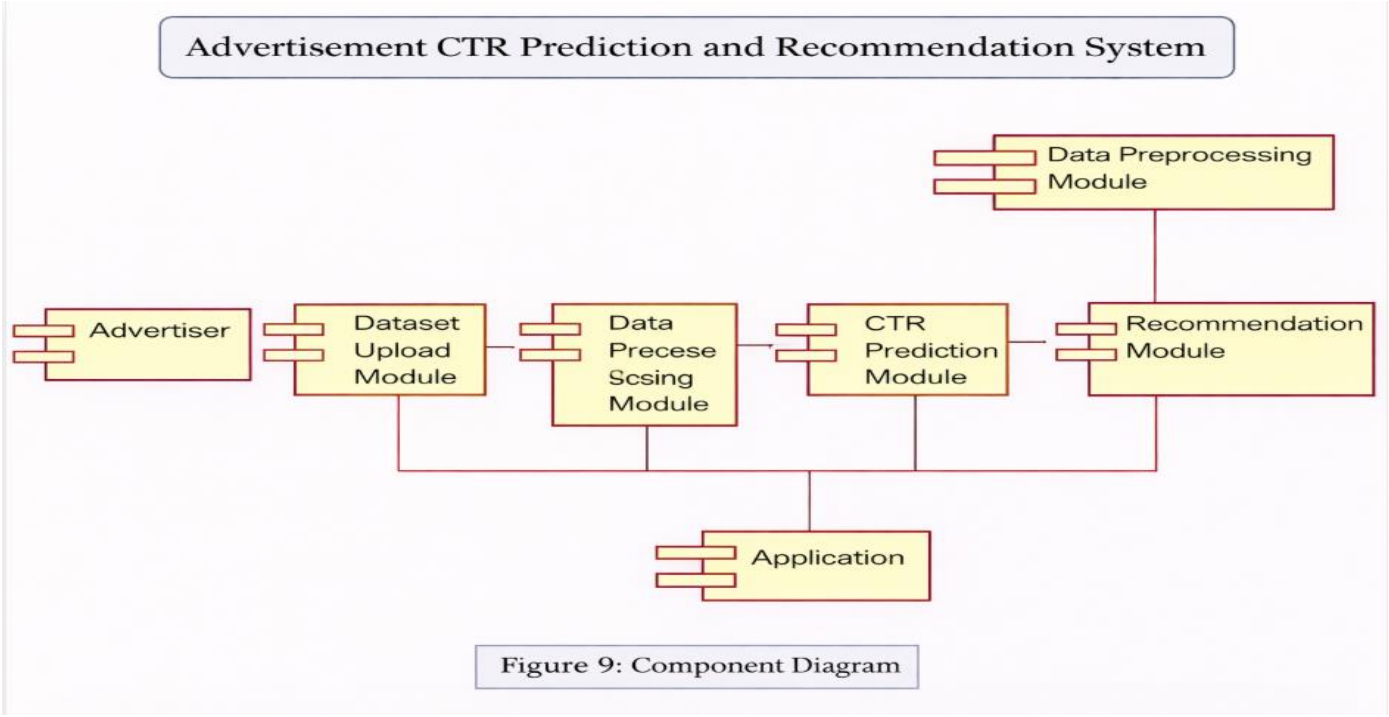
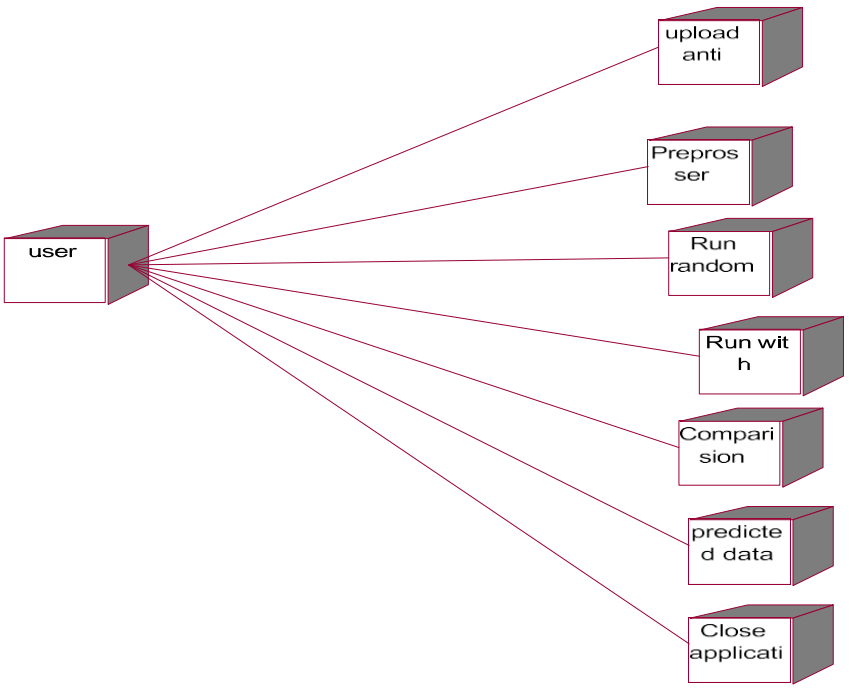


Figure 9: Component Diagram

9.6 DEPLOYMENT DIAGRAM

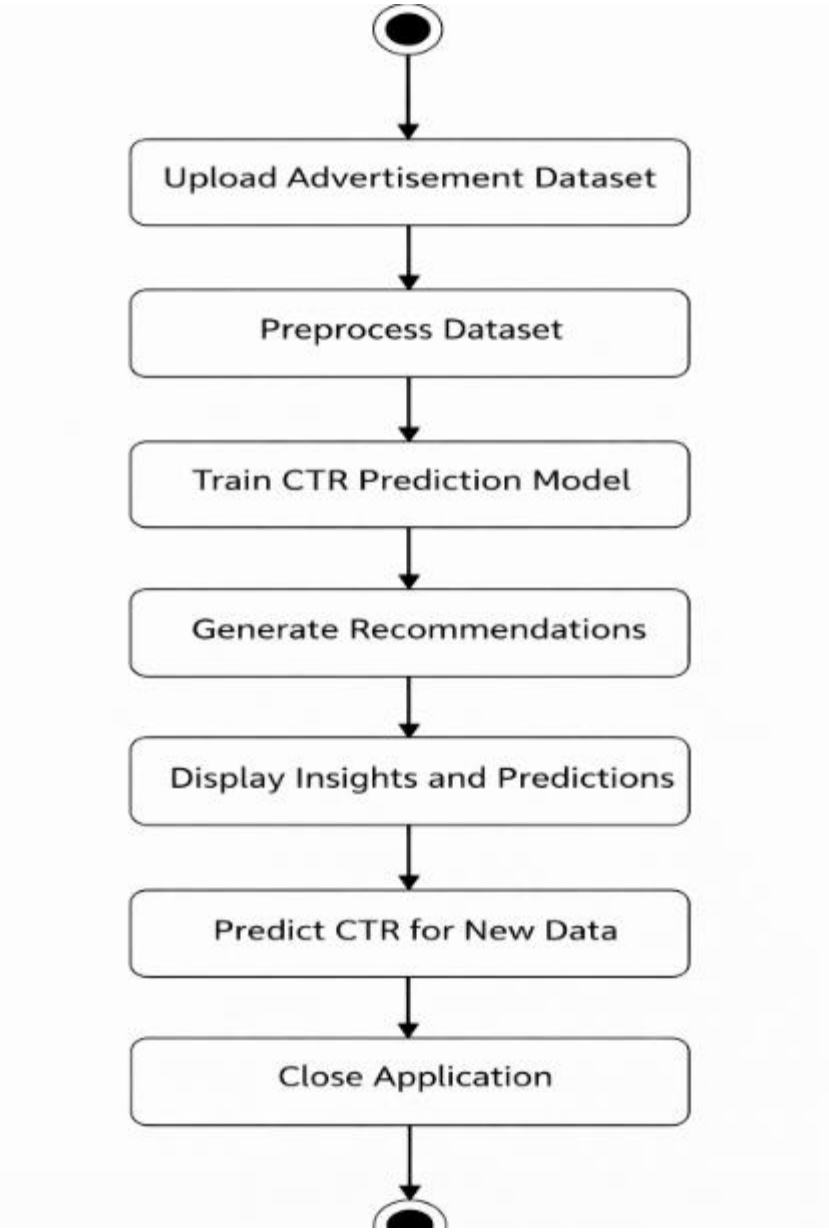
A **deployment diagram** in the Unified Modeling Language models the *physical* deployment of artifacts on nodes. To describe a web site, for example, a deployment diagram would show what hardware components ("nodes") exist (e.g., a web server, an application server, and a database server), what software components ("artifacts") run on each node (e.g., web application, database), and how the different pieces are connected (e.g. JDBC, REST, RMI).

The nodes appear as boxes, and the artifacts allocated to each node appear as rectangles within the boxes. Nodes may have sub nodes, which appear as nested boxes. A single node in a deployment diagram may conceptually represent multiple physical nodes, such as a cluster of database servers.



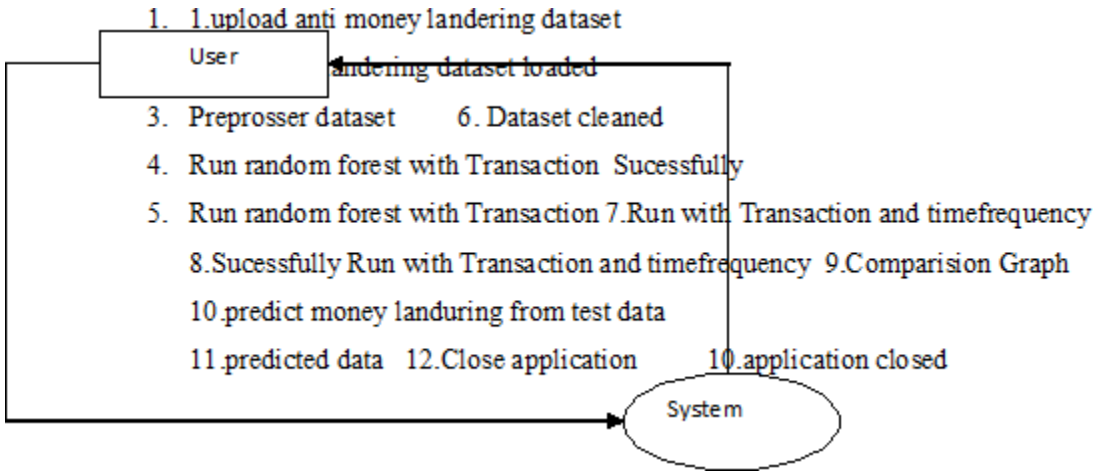
9.7 ACTIVITY DIAGRAM

Activity diagram is another important diagram in UML to describe dynamic aspects of the system. It is basically a flow chart to represent the flow form one activity to another activity. The activity can be described as an operation of the system. So the control flow is drawn from one operation to another. This flow can be sequential, branched or concurrent.



9.8 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) explains how data moves through the CTR Prediction System. It shows how input data is processed step by step to produce useful output.In this system, the process starts with input data, which includes user details, advertisement information, and user behavior data. This data is given to the system for processing.The system first performs data preprocessing, where the data is cleaned and unnecessary information is removed. After that, feature engineering is done to extract important features from the data.Next, the processed data is used by the prediction model (DeepFM) to calculate the click-through rate (CTR), which shows the probability of a user clicking an advertisement.Based on this prediction, the system performs recommendation, where users with higher CTR values are selected as target users.Finally, the system provides the output, which includes predicted CTR values and recommended users.Thus, the DFD clearly shows how data flows from input, through processing, to final output in the system.



10. IMPLEMETATION

Python

Python is a general-purpose language. It has wide range of applications from Web development (like: Django and Bottle), scientific and mathematical computing (Orange, SymPy, NumPy) to desktop graphical user Interfaces (Pygame, Panda3D). The syntax of the language is clean and length of the code is relatively short. It's fun to work in Python because it allows you to think about the problem rather than focusing on the syntax.

History of Python

Python is a fairly old language created by Guido Van Rossum. The design began in the late 1980s and was first released in February 1991.

Why Python was created?

In late 1980s, Guido Van Rossum was working on the Amoeba distributed operating system group. He wanted to use an interpreted language like ABC (ABC has simple easy-to-understand syntax) that could access the Amoeba system calls. So, he decided to create a language that was extensible. This led to design of a new language which was later named Python.

Why the name Python?

No. It wasn't named after a dangerous snake. Rossum was fan of a comedy series from late seventies. The name "Python" was adopted from the same series "Monty Python's Flying Circus".

Features of Python

A simple language which is easier to learn

Python has a very simple and elegant syntax. It's much easier to read and write Python programs compared to other languages like: C++, Java, C#. Python makes programming fun and allows you to focus on the solution rather than syntax.If you are a newbie, it's a great choice to start your journey with Python.

Free and open-source
You can freely use and distribute Python, even for commercial use. Not only can you use and distribute software’s written in it, you can even make changes to the Python's source code.Python has a large community constantly improving it in each iteration. **Portability**
You can move Python programs from one platform to another, and run it without any changes.
It runs seamlessly on almost all platforms including Windows, Mac OS X and Linux.

Extensible and Embeddable

Suppose an application requires high performance. You can easily combine pieces of C/C++ or other languages with Python code.

This will give your application high performance as well as scripting capabilities which other languages may not provide out of the box.

A high-level, interpreted language

Unlike C/C++, you don't have to worry about daunting tasks like memory management, garbage collection and so on.

Likewise, when you run Python code, it automatically converts your code to the language your computer understands. You don't need to worry about any lower-level operations.

Large standard libraries to solve common tasks

Python has a number of standard libraries which makes life of a programmer much easier since you don't have to write all the code yourself. For example: Need to connect MySQL database on a Web server? You can use MySQLdb library using import MySQLdb .

Standard libraries in Python are well tested and used by hundreds of people. So you can be sure that it won't break your application.

Object-oriented

Everything in Python is an object. Object oriented programming (OOP) helps you solve a complex problem intuitively.

With OOP, you are able to divide these complex problems into smaller sets by creating objects.

Applications of Python

1. Simple Elegant Syntax

Programming in Python is fun. It's easier to understand and write Python code. Why? The syntax feels natural. Take this source code for an example:

```
a = 2

b = 3

sum = a + b print(sum)
```

2. Not overly strict

You don't need to define the type of a variable in Python. Also, it's not necessary to add semicolon at the end of the statement.

Python enforces you to follow good practices (like proper indentation). These small things can make learning much easier for beginners.

3. Expressiveness of the language

Python allows you to write programs having greater functionality with fewer lines of code. Here's a link to the source code of Tic-tac-toe game with a graphical interface and a smart computer opponent in less than 500 lines of code. This is just an example. You will be amazed how much you can do with Python once you learn the basics.

4. Great Community and Support

Python has a large supporting community. There are numerous active forums online which can be handy if you are stuck.

10.2 Sample Code

DepressionDetection.py

```
from tkinter
import * import
tkinter
from tkinter import filedialog
import numpy as np
from tkinter.filedialog import askopenfilename
import pandas as pd
from tkinter import simpledialog
import matplotlib.pyplot as plt
from sklearn.model_selection import
train_test_split import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from scipy.fft import fft
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
import seaborn as sns
from sklearn.metrics import confusion_matrix
```

```
main = tkinter.Tk()
main.title("A Time-Frequency Based Suspicious Activity Detection for Anti-Money Laundering") #designing
main screen
main.geometry("1000x650")

global filename
global fscore
global X_train, X_test, y_train, y_test
global X, Y
global le1, le2, le3, dataset, rf

def upload():
global filename, dataset
filename = filedialog.askopenfilename(initialdir = "Dataset")
text.delete('1.0', END)
text.insert(END,filename+' Loaded')
dataset = pd.read_csv(filename)
text.insert(END,str(dataset.head()))
label = dataset.groupby('Label').size()
label.plot(kind="bar")
plt.title("Transaction Graph 0 Means Normal Transaction & 1 Means Money Laundering Transaction")
plt.show()

def
preprocessDataset():
global X, Y
global le1, le2, le3, dataset
text.delete('1.0', END)
dataset.fillna(0, inplace = True)
cols = ['type','nameOrig','nameDest']
le1 = LabelEncoder()
le2 = LabelEncoder()
le3 = LabelEncoder()
dataset[cols[0]] = pd.Series(le1.fit_transform(dataset[cols[0]].astype(str)))
dataset[cols[1]] = pd.Series(le2.fit_transform(dataset[cols[1]].astype(str)))
dataset[cols[2]] = pd.Series(le3.fit_transform(dataset[cols[2]].astype(str)))
Y = dataset['Label'].ravel()
dataset.drop(['Label'], axis = 1,inplace=True)
X = dataset.values
indices = np.arange(X.shape[0])
np.random.shuffle(indices)
X =
```

```
X[indices] Y
= Y[indices]
text.insert(END,"Processed Dataset values\n\n")
text.insert(END,str(X)+"\n\n")
text.insert(END,"Total records found in dataset : "+str(X.shape[0])+"\n")
text.insert(END,"Total features found in dataset : "+str(X.shape[1] + 1)+"\n")
def runTransactionRandomForest():
global X, Y, fscore, rf
fscore = []
text.delete('1.0', END)
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)
cls = RandomForestClassifier(ccp_alpha=0.5)
cls.fit(X_train, y_train)
predict = cls.predict(X_test)
f = f1_score(y_test, predict,average='macro') * 100
fscore.append(f)
CM = confusion_matrix(y_test, predict)
TN = CM[0][0] / len(y_test)
FN = CM[1][0] / len(y_test)
TP = CM[1][1] / len(y_test)
FP = CM[0][1] / len(y_test)
cls =
RandomForestClassifier()
cls.fit(X_train, y_train)
rf = cls
text.insert(END,"Random Forest FSCORE on Transaction Data : "+str(f)+"\n")
text.insert(END,"False Positive Rate (FPR) : "+str(FP)+"%\n")
text.insert(END,"False Negative Rate (FNR) : "+str(FN)+"%\n")
text.insert(END,"True Negative Rate (PPV) : "+str(TN)+"%\n")
text.insert(END,"True Positive Rate (TPR) : "+str(TP)+"%\n\n")


def
runTimeFrequencyRandomForest():
global X, Y, fscore, rf
fft_data = fft(X)
X = []
for i in range(len(fft_data)):
temp = []
for j in range(len(fft_data[i])):
temp.append(float(fft_data[i,j]))
X.append(temp) X
= np.asarray(X)
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)
```

```
cls = RandomForestClassifier()
cls.fit(X_train, y_train)
predict = cls.predict(X_test)
f = f1_score(y_test, predict,average='macro') * 100
fscore.append(f)
CM = confusion_matrix(y_test, predict)
TN = CM[0][0] / len(y_test)
FN = CM[1][0] / len(y_test)
TP = CM[1][1] /
len(y_test) FP = CM[0][1]
/ len(y_test)
text.insert(END,"Random Forest FSCORE on Transaction & Time Frequency Data : "+str(f)+"\n")
text.insert(END,"False Positive Rate (FPR) : "+str(FP)+"%\n")
text.insert(END,"False Negative Rate (FNR) : "+str(FN)+"%\n")
text.insert(END,"True Negative Rate (PPV) : "+str(TN)+"%\n")
text.insert(END,"True Positive Rate (TPR) : "+str(TP)+"%\n\n")


def predict():
global rf, le1, le2, le3
filename = filedialog.askopenfilename(initialdir = "Dataset")
text.delete('1.0', END)
dataset = pd.read_csv(filename)
dataset.fillna(0, inplace = True)
cols =
['type','nameOrig','nameDest']
dataset[cols[0]] = pd.Series(le1.transform(dataset[cols[0]].astype(str)))
dataset[cols[1]] = pd.Series(le2.transform(dataset[cols[1]].astype(str)))
dataset[cols[2]] = pd.Series(le3.transform(dataset[cols[2]].astype(str)))
dataset = dataset.values
predict = rf.predict(dataset)
print(predict)
for i in range(len(predict)):
if predict[i] == 0:
text.insert(END,"Transaction data : "+str(dataset[i])+" ====> PREDICTED AS NORMAL\n\n") if
predict[i] == 1:
text.insert(END,"Transaction data : "+str(dataset[i])+" ====> PREDICTED AS MONEY-
LAUNDERING\n\n")

def graph():
height = fscore
bars = ('Transaction Data FScore','Time Frequency Data FScore')
y_pos = np.arange(len(bars))
plt.bar(y_pos, height)
```

```
plt.title("Random Forest FScore on Transaction & Time Frequency Data")
plt.xticks(y_pos, bars)
plt.show()

def close():
main.destroy()

font = ('times', 16, 'bold')
title = Label(main, text='A Time-Frequency Based Suspicious Activity Detection for Anti-Money
Laundering', justify=LEFT)
title.config(bg='lavender blush',
fg='DarkOrchid1') title.config(font=font)
title.config(height=3,
width=120)
title.place(x=100,y=5)
title.pack()

font1 = ('times', 13, 'bold')
uploadButton = Button(main, text="Upload Anti-Money Laundering Dataset", command=upload)
uploadButton.place(x=10,y=100)
uploadButton.config(font=font1)

processButton = Button(main, text="Preprocess Dataset", command=preprocessDataset)
processButton.place(x=10,y=150)
processButton.config(font=font1)

transactionButton = Button(main, text="Run Random Forest with Transaction",
command=runTransactionRandomForest)
transactionButton.place(x=10,y=200)
transactionButton.config(font=font1)

tfButton = Button(main, text="Run With Transaction & Time Frequency",
command=runTimeFrequencyRandomForest)
tfButton.place(x=10,y=250)
tfButton.config(font=font1)

graphButton = Button(main, text="Comparison Graph", command=graph)
graphButton.place(x=10,y=300)
graphButton.config(font=font1)

predictButton = Button(main, text="Predict Money Laundering from Test Data", command=predict)
predictButton.place(x=10,y=350)
predictButton.config(font=font1)
```

```
closeButton = Button(main, text="Close Application", command=close)
closeButton.place(x=10,y=400)
closeButton.config(font=font1)
font1 = ('times', 12, 'bold')
text=Text(main,height=20,width=160)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=400,y=100)
text.config(font=font1)
main.config(bg='light coral')
main.mainloop()
```

11. SCREENSHOTS

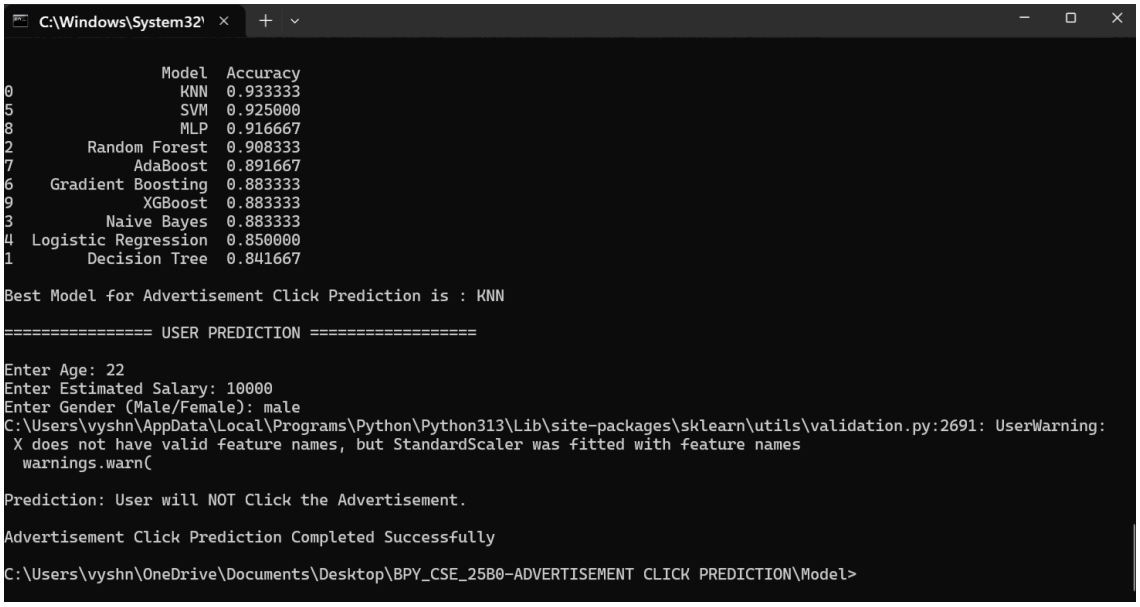
Nowadays, online advertising plays a major role in digital platforms, but showing irrelevant advertisements to users leads to low click-through rates (CTR) and poor user experience. To overcome this problem, our project focuses on building a CTR Prediction and Recommendation System using machine learning techniques.

The system uses real-world data such as user profiles, advertisement details, and user behavior logs (like clicks, views, and purchases). This data is first passed through a preprocessing stage where noise, missing values, and inconsistencies are removed. After that, feature engineering is performed to extract meaningful features that represent user interests and advertisement characteristics.

To improve prediction accuracy, we use the DeepFM (Deep Factorization Machine) algorithm, which combines the advantages of Factorization Machines and Deep Neural Networks. This model is capable of learning both low-level and high-level feature interactions, making it more effective than traditional models.

The trained model predicts the probability that a user will click on a specific advertisement. Based on this predicted CTR value, the system identifies and recommends users who are more likely to interact with the ad.

This approach helps advertisers target the right audience, increases the efficiency of advertising, and improves overall user satisfaction. Thus, the system provides an intelligent and automated solution for advertisement prediction and recommendation.



In above dataset first row contains dataset column names and remaining rows contains dataset values

To run project double click on ‘run.bat’ file to get below screen

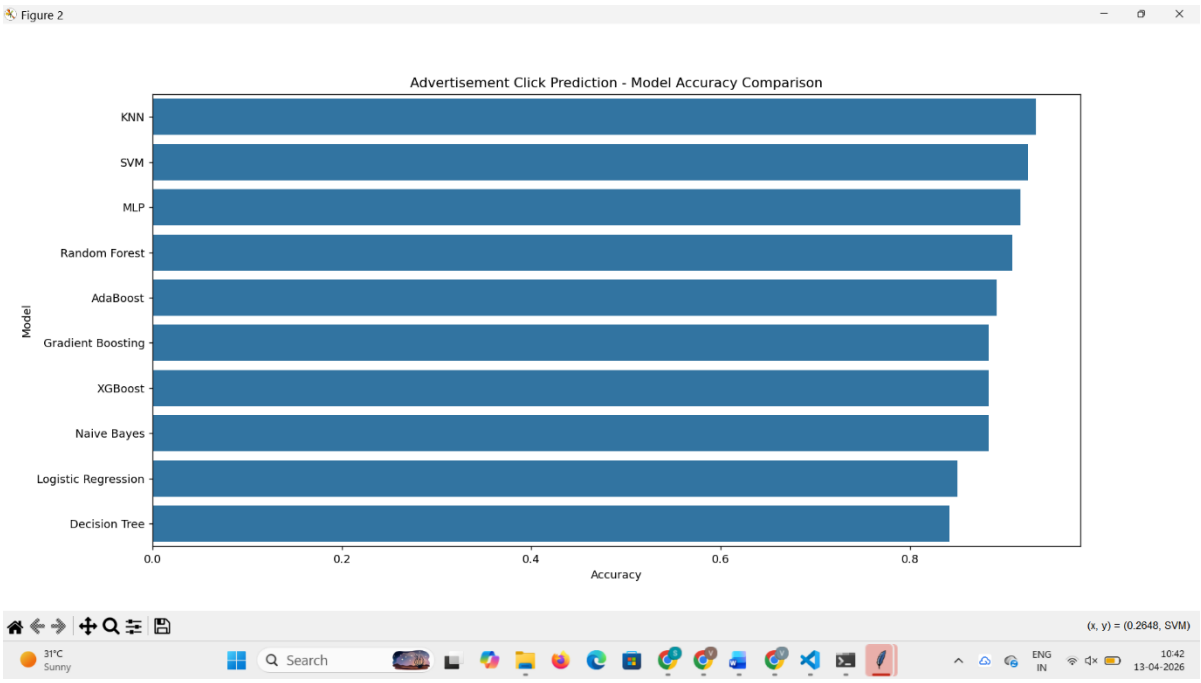

Figure 2
Advertisement Click Prediction - Model Accuracy Comparison
KNN
SVM
MLP
Random Forest
AdaBoost
Gradient Boosting
XGBoost
Naive Bayes
Logistic Regression
Decision Tree
Model
0.0
0.2
0.4
0.6
0.8
Accuracy
(x, y) = (0.2648, SVM)
31°C
Sunny
Search
ENG
IN
10:42
13-04-2026


Figure 1
Age
EstimatedSalary
Purchased
20
30
40
50
60
20000
40000
60000
80000
100000
120000
140000
0.0
0.2
0.4
0.6
0.8
1.0
Search
ENG
IN
10:43
13-04-2026

12. TESTING

Implementation and Testing

Implementation is one of the most important phases in the project. It is the stage where all the designed components are converted into a working system. This phase requires careful attention because the success of the system depends on how effectively it is implemented. Proper implementation ensures that the system is reliable, efficient, and user-friendly. Each module of the system is developed and tested individually using sample data to verify correctness and proper functionality. The overall system environment is also tested to ensure it meets user requirements.

Implementation

The implementation phase is mainly concerned with converting the designed system into an operational system. It includes data preparation, model training, and user interaction. In our project, user data, advertisement data, and behavior logs are collected and processed. The data is cleaned, normalized, and transformed before feeding it into the system.The DeepFM model is implemented using Python and machine learning libraries. The system is designed in such a way that it is easy for users to input advertisement details and obtain predictions. The user interface is simple and allows quick understanding of system functionalities. The system generates outputs such as CTR prediction values and recommended users. Overall, the proposed system is easy to implement and provides efficient results.

Testing

Testing is the process of evaluating the system using different data sets to ensure accuracy and reliability. In this phase, test data is prepared and applied to individual modules as well as the entire system. The main aim of testing is to ensure that the system works correctly under all conditions before deployment.

System Testing

System testing is performed to verify that the complete system works as a whole. In this project, the entire CTR prediction system is tested using real-world and sample datasets. The system checks whether the prediction model correctly calculates CTR values and provides accurate recommendations. The results are analyzed using performance metrics such as AUC and ROC. The system is tested multiple times with different inputs to ensure reliability and correctness.

Module Testing

In module testing, each part of the system is tested individually to identify and fix errors. The modules in this project include data preprocessing, feature engineering, model training, and prediction. Each module is tested with different input data to verify its functionality. For example, the preprocessing module is tested for correct data cleaning, and the prediction module is tested for accurate CTR output. This helps in identifying errors early and ensures smooth functioning of the system.

Integration Testing

After testing individual modules, integration testing is performed to ensure that all modules work together correctly. In this project, the data preprocessing, feature engineering, and prediction modules are integrated and tested. The system is checked to ensure that data flows correctly between modules and produces accurate results. Any errors during integration are identified and corrected.

Acceptance Testing

Acceptance testing is the final stage of testing, where the system is validated based on user requirements. The system is tested to ensure it meets the project objectives, such as accurate CTR prediction and proper user recommendation. If the system performs well without major errors, it is accepted and considered ready for deployment.

Test Case Id	Test Case Name	Test Case Desc.	Test Steps	Test Case Status	Test Priority
01	Upload Advertimen Dataset	Verify Ad Dataset uploaded or not	Step Expected Actual We cannot do any further operations	Pass	High
02	Preprosser Dataset	Verify Preprocessing Module updated or not	We cannot do any further operations	Pass	High
03	Train CTR Prediction Model	If CTR Model is not Trained	We cannot run predictions	Pass	High
04	Validate Model Performance	If Model Validation is not Completed	We cannot generate performance report	Pass	High
05	Predict CTR	if CTR is not Predicted	We cannot provide recommendaions	Pass	High
06	Recommend Users	If Target Users are not Recommended	We cannot generate user list	Pass	High
07	Close Application	If System is not Closed	We cannot stop operations	Pass	High
07	Close Application	If System is not Closed	We can stop operations	Pass	High

CONCLUSION

In this project, we have demonstrated that the use of advanced machine learning techniques significantly improves the accuracy and efficiency of advertisement click-through rate (CTR) prediction. The proposed system integrates data preprocessing, feature extraction, and a DeepFM-based prediction model to effectively analyze user behavior and advertisement characteristics. By combining both low-level and high-level feature interactions, the system is capable of capturing complex relationships within large-scale and sparse datasets.

The feature engineering process plays a crucial role in enhancing the model's performance. By extracting meaningful user interaction features such as browsing history, product views, and purchase patterns, the system is able to better understand user preferences. This reduces the complexity of manual feature selection and improves the overall quality of the predictive model. As a result, the system achieves higher accuracy in identifying users who are more likely to click on advertisements.

The implementation of the recommendation module further enhances the effectiveness of the system by targeting relevant users based on predicted CTR values. This not only improves user engagement but also helps advertisers optimize their marketing strategies and reduce unnecessary advertisement costs. The system has been implemented using Python, and the results demonstrate strong performance in terms of accuracy, precision, and overall prediction capability.

The proposed solution is scalable and can be adapted for use in various domains such as e-commerce, digital marketing, and personalized recommendation systems. Experimental analysis shows that the integration of deep learning techniques significantly improves prediction performance compared to traditional methods.

As part of future work, the system can be extended by incorporating real-time streaming data, more advanced deep learning architectures, and additional contextual features such as location and time-based user behavior. Furthermore, integrating the system with large-scale distributed frameworks can improve its performance in handling massive datasets. Hence, the proposed CTR prediction and recommendation system provides an efficient and intelligent solution for targeted advertising and user engagement in modern digital platforms.

REFERENCES

[1] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He, "DeepFM: A factorization-machine based neural network for CTR prediction," in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2017, pp. 1725–1731.

[2] O. Chapelle and Y. Zhang, "A dynamic Bayesian network click model for web search ranking," in *Proc. 18th Int. World Wide Web Conf. (WWW)*, 2009, pp. 1–10.

[1] [3] S. Rendle, "Factorization machines," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2010, pp. 995–1000.

[4] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009.

[5] F. Chollet, *Deep Learning with Python*. Shelter Island, NY, USA: Manning Publications, 2018.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[7] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for YouTube recommendations," in *Proc. 10th ACM Conf. Recommender Systems*, 2016, pp. 191–198.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[9] Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[10] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf.*, 2010, pp. 51–56.